10 Easy Python Case Studies

Beginner-friendly solutions using basic Python topics from the assigned syllabus.

Note: The numbers and policies below are examples for practice. Change them to match the exact rules given by your teacher or case study.

1. University Scholarship Allocation

Concepts: variables, data types, operators, conditions, loops, lists and tuples.

```
# Eligible students are ranked by marks, then attendance.
students = [
    ("Anu", 92, 80, 95, 150000),
    ("Ravi", 85, 75, 90, 100000),
    ("Maya", 92, 82, 95, 120000),
    ("Kiran", 78, 70, 88, 90000)
]
eligible = []

for name, marks, subject_min, attendance, income in students:
    if marks >= 80 and subject_min >= 70 and attendance >= 75 and income <= 200000:
        eligible.append((name, marks, attendance))

eligible.sort(key=lambda x: (x[1], x[2]), reverse=True)

print("Scholarship ranking:")
for rank, student in enumerate(eligible, 1):
    print(rank, student[0], "Marks:", student[1], "Attendance:", student[2])
```

2. Progressive Electricity Billing

Concepts: variables, data types, operators, conditions, loops, lists and tuples.

```
# Example slabs: first 100 units, next 100, above 200.
monthly_units = [85, 130, 240]
bills = []

for units in monthly_units:
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5

    fixed_charge = 50
    surcharge = energy * 0.05
    bill = energy + fixed_charge + surcharge
    bills.append(bill)
    print("Units:", units, "Bill: Rs.", round(bill, 2))

for i in range(1, len(bills)):
    if bills[i] > bills[i - 1] * 1.25:
        print("Unusual increase in month", i + 1)
```

3. Hospital Billing and Patient Classification

Concepts: variables, data types, operators, conditions, loops, lists and tuples.

```
# Charges are sample values; change them for your policy.
patients = [
    ("Patient A", 500, 1200, 3000, 65),
    ("Patient B", 400, 800, 1500, 20)
]

for name, consultation, tests, treatment, age in patients:
    total = consultation + tests + treatment

    if age >= 60:
        concession = total * 0.10
    else:
        concession = 0

    final_bill = total - concession

    if final_bill <= 2000:
        category = "Low payment"
    elif final_bill <= 5000:
        category = "Medium payment"
    else:
        category = "High payment"

    print(name, "Total:", total, "Concession:", concession)
    print("Final bill:", final_bill, "Category:", category)
```

4. Employee Payroll and Deduction Analysis

Concepts: variables, data types, operators, conditions, loops, lists and tuples.

```
employees = [
    ("Anu", 20000, 3000, 1000, 1500),
    ("Ravi", 25000, 4000, 2000, 1000)
]

for name, basic, allowance, overtime, incentive in employees:
    gross = basic + allowance + overtime + incentive

    # Sample progressive deduction policy
    if gross <= 20000:
        deduction = gross * 0.05
    elif gross <= 30000:
        deduction = 20000 * 0.05 + (gross - 20000) * 0.10
    else:
        deduction = 20000 * 0.05 + 10000 * 0.10 + (gross - 30000) * 0.15

    net_salary = gross - deduction
    print("\nEmployee:", name)
    print("Gross salary: Rs.", round(gross, 2))
    print("Deduction: Rs.", round(deduction, 2))
    print("Net salary: Rs.", round(net_salary, 2))
```

5. Multi-Plan Telecommunications Cost Optimization

Concepts: variables, data types, operators, conditions, loops, lists and tuples.

```
# Plan format: (name, included_minutes, monthly_price)
plans = [
    ("Basic", 100, 199),
    ("Plus", 300, 349),
    ("Max", 600, 499)
]
usage = 260
budget = 400
choices = []

for name, included, price in plans:
    extra_minutes = max(0, usage - included)
    total = price + extra_minutes * 1.5

    if total <= budget:
        choices.append((total, name, extra_minutes))

if choices:
    choices.sort()
    cost, name, extra = choices[0]
    print("Best plan:", name)
    print("Total cost: Rs.", round(cost, 2))
    print("Extra minutes:", extra)
else:
    print("No plan fits the budget.")
```

6. Railway Fare and Reservation Calculator

Concepts: variables, data types, operators, conditions, loops, lists and tuples.

```
passengers = [
    ("Asha", 250, "Sleeper", 12),
    ("Ravi", 400, "AC", 35),
    ("Mohan", 300, "Sleeper", 67)
]

for name, distance, travel_class, age in passengers:
    if travel_class == "Sleeper":
        rate = 1.0
    else:
        rate = 2.0

    fare = distance * rate
    concession = 0

    if age < 12:
        concession = fare * 0.50
    elif age >= 60:
        concession = fare * 0.30

    final_fare = fare - concession + 30  # reservation fee
    print(name, "Fare: Rs.", round(final_fare, 2))

# Cancellation example: 20% penalty on a Rs. 500 fare
fare = 500
penalty = fare * 0.20
print("Cancellation refund: Rs.", fare - penalty)
```

7. Retail Billing with Multiple Discount Rules

Concepts: variables, data types, operators, conditions, loops, lists and tuples.

```
# Product format: (name, category, price, quantity)
products = [
    ("Shirt", "clothing", 800, 2),
    ("Book", "books", 300, 3),
    ("Bag", "other", 600, 1)
]
subtotal = 0
tax_total = 0

for name, category, price, quantity in products:
    amount = price * quantity

    if category == "clothing":
        tax_rate = 0.05
    elif category == "books":
        tax_rate = 0.02
    else:
        tax_rate = 0.10

    tax = amount * tax_rate
    subtotal += amount
    tax_total += tax
    print(name, "Amount:", amount, "Tax:", round(tax, 2))

discount = subtotal * 0.10 if subtotal >= 2000 else 0
discount = min(discount, 500)  # maximum discount
total = subtotal + tax_total - discount

print("Subtotal:", subtotal)
print("Tax:", round(tax_total, 2))
print("Discount:", round(discount, 2))
print("Final invoice: Rs.", round(total, 2))
```

8. Manufacturing Cost and Profitability Analysis

Concepts: variables, data types, operators, conditions, loops, lists and tuples.

```
# Sample production details
material_cost = 12000
labour_cost = 5000
machine_cost = 2500
wastage_rate = 0.05
units = 100
selling_price_per_unit = 230
target_profit = 2000

wastage_cost = material_cost * wastage_rate
total_cost = material_cost + labour_cost + machine_cost + wastage_cost
selling_total = units * selling_price_per_unit
profit = selling_total - total_cost

print("Wastage cost:", round(wastage_cost, 2))
print("Total production cost:", round(total_cost, 2))
print("Total sales:", selling_total)
print("Profit:", round(profit, 2))

if profit >= target_profit:
    print("Target profit achieved.")
else:
    print("Target profit not achieved.")
```

9. Multi-Currency Travel Budget Analysis

Concepts: variables, data types, operators, conditions, loops, lists and tuples.

```
# Expenses: (category, amount in foreign currency, exchange rate)
expenses = [
    ("Hotel", 200, 83.0),
    ("Food", 80, 83.0),
    ("Transport", 50, 83.0),
    ("Shopping", 100, 83.0)
]
budget = 30000
fee_rate = 0.02
total_inr = 0
largest_category = ""
largest_amount = 0

for category, amount, rate in expenses:
    converted = amount * rate
    fee = converted * fee_rate
    total = converted + fee
    total_inr += total
    print(category, "Cost in INR:", round(total, 2))

    if total > largest_amount:
        largest_amount = total
        largest_category = category

print("Total spending: Rs.", round(total_inr, 2))
print("Largest expense:", largest_category)

if total_inr > budget:
    print("Budget exceeded by Rs.", round(total_inr - budget, 2))
else:
    print("Budget remaining: Rs.", round(budget - total_inr, 2))
```

10. Campus Event Financial Planning

Concepts: variables, data types, operators, conditions, loops, lists and tuples.

```
venue = 12000
equipment = 5000
catering_per_person = 150
transport_per_person = 40
certificate_per_person = 10
sponsorship = 8000
registration_fee = 300
participants = 100

fixed_cost = venue + equipment
variable_per_person = catering_per_person + transport_per_person + certificate_per_person
total_cost = fixed_cost + variable_per_person * participants
income = sponsorship + registration_fee * participants
balance = income - total_cost

print("Total cost: Rs.", total_cost)
print("Total income: Rs.", income)
print("Surplus / shortfall: Rs.", balance)

# Minimum participants needed to break even
needed = 0
while sponsorship + registration_fee * needed < fixed_cost + variable_per_person * needed:
    needed += 1

print("Minimum registrations to break even:", needed)
```